

作業系統期中考 —— 歷屆考題與解答整理

2011–2016 (依主題分類)

說明：同一題型跨年出現時，僅列最完整版本並標註出現年份，避免重複。

壹、中斷與 I/O 機制

【題型 1】Interrupt-based vs Busy-waiting (每年必考)

題目 (2011/2013/2014/2015 皆考) : Show the advantage of interrupt-based operations over busy-waiting.

標準答案：

- **Busy-waiting (輪詢)** : CPU 必須持續在迴圈中輪詢設備是否就緒，在等待期間無法做其他事，大量浪費 CPU 時間，CPU 幾乎處於閒置狀態。
 - **Interrupt-based (中斷驅動)** : CPU 啟動 I/O 後繼續執行其他程式，等到設備完成才發送中斷信號。CPU 停止當前工作並轉移控制給 interrupt handler 處理，完成後繼續執行。CPU 始終保持忙碌，使用率高。
-

【題型 2】硬體中斷處理流程 (每年必考)

題目 (2011/2012/2013/2014/2015/2016) : Describe the process of hardware interrupt handling. You need to mention device controller, interrupt-request line, state saving, interrupt vector, interrupt-specific handler.

標準答案：

1. **Device controller** 在其 local buffer 中儲存資料，I/O 完成後在 **interrupt-request line** 上發出中斷信號。
 2. CPU 在執行完每條指令後都會偵測 interrupt-request line。
 3. CPU 接收到中斷，儲存當前行程的狀態 (state saving，存入 PCB/registers)。
 4. CPU 轉移到一個通用的 interrupt-handling routine，再使用中斷號碼查詢 **interrupt vector (中斷向量表)**，找到對應的 interrupt-specific handler 位址。
 5. 執行對應的 **interrupt-specific handler** 處理中斷 (例如：將 local buffer 的資料複製到 main memory)。
 6. 執行 return-from-interrupt 指令，恢復被中斷行程的狀態，CPU 繼續執行。
-

【題型 3】中斷驅動 I/O 七步驟排列 (每年必考)

題目 (2015/2016 有給 flowchart 填空) : The interrupt-driven I/O cycle consists of the following steps. Organize the steps with the flowchart.

七個步驟：

- (i) Input ready, output complete, or error generates interrupt signal
- (ii) Interrupt handler processes data, return from interrupt
- (iii) CPU executing, checks for interrupts between instructions
- (iv) Device driver initiates I/O
- (v) Controller initiates I/O
- (vi) CPU resumes processing of interrupted process
- (vii) CPU receiving interrupt, transfer control to interrupt handler

標準答案 (**flowchart** 位置)：



【題型 4】Interrupt Vector 的好處 (2011/2014 考)

題目：Explain the benefit of using interrupt vector.

標準答案：使用 interrupt vector，CPU 可以用中斷號碼直接查表 (IVT) 找到對應的 interrupt-specific handler 的程式碼位址，不需要逐一搜尋所有 **interrupt handlers**，大幅加速中斷處理速度，並支援多種不同類型的中斷服務。

【題型 5】Device Controller vs Device Driver (2013/2015 考)

題目：What role do device controllers and device drivers play in a computer system?

標準答案：

- **Device controller (硬體)**：讀取或寫入儲存在設備中的資料，有自己的 local buffer，I/O 完成後透過 interrupt-request line 通知 CPU。
- **Device driver (軟體)**：OS 中的軟體元件，告訴作業系統如何與 device controller 溝通 (提供統一介面)。Device drivers send requests to device controllers; if the controller receives the request, it asks the device to do I/O operations.

貳、系統結構 (System Structures)

【題型 6】API 呼叫 System Call (每年必考)

題目 (2011/2012/2013/2014/2015/2016) : How does an API function invoke a system call? You **need to mention** software interrupt, dual-mode, system call number (index), and table of code pointer.

標準答案 (必答四要素) :

在 user mode 中，API 函數做以下事情：

1. API 函數觸發一個 **software interrupt (trap)**，這讓 CPU 切換到 **kernel mode (dual-mode 切換)** 來處理緊急情況。
2. API 函數將對應的 **system call number (index)** 放入 EAX 暫存器，這個號碼代表要執行的系統服務。
3. CPU 使用此 index 查詢 **table of code pointer (系統呼叫表)**，找到對應 kernel 函數的位址。
4. 執行該 kernel 函數 (在 kernel space 執行 system call)。
5. 完成後，CPU 切回 **user mode**，恢復程式計數器，繼續行程執行。

【題型 7】為何偏好 API 而非直接呼叫 System Call (每年必考)

題目：Explain why programmers prefer programming according to API functions rather than invoking system calls directly.

標準答案：

1. 可攜性 (**Portability**) : API 函數提供跨平台的一致介面，可在不同作業系統編譯執行 (例如 POSIX API 在所有 UNIX 系統通用) ；若直接呼叫 system call，不同 OS 的 call number 不同，程式無法移植。
2. 簡便性 (**Ease of use**) : 直接呼叫 system call 繁瑣且困難，API 提供更簡單、更直覺的介面，讓程式設計師不需了解底層 system call 的細節即可完成工作。

【題型 8】CLI 兩種實作方式比較 (2015 考)

題目：There are two different ways that commands can be processed by a command interpreter. Compare and contrast the two approaches.

標準答案：

	方式一：命令內建	方式二：系統程式 (UNIX)
做法	指令碼直接放在直譯器中	直譯器搜尋 PATH，找到對應系統程式後載入執行
優點	執行速度快	直譯器體積小；可輕易新增指令
缺點	直譯器體積大；少用指令浪費記憶體；新增指令需修改直譯器	執行指令需要搜尋和載入，速度較慢

Most commands in shell are to implement file system. If a command interpreter contains all commands, the interpreter size will be huge but quick. Some commands are rare to use and loaded in memory will be waste. With system programs, interpreter can be small in size but takes a longer time to execute a command.

【題型 9】高階語言實作 OS 的優點 (2015 考)

題目：What are the advantages of using a higher-level language to implement an operating system?

標準答案：

1. 人類容易閱讀和理解程式碼 (easy to read and understand) 。
2. 程式碼可以更快速撰寫 (code can be written faster) 。
3. 程式設計師可以與隊友分享程式碼，使用 compiler 和 debugger 進行除錯，開發更多模組。
4. **OS 更容易移植到其他硬體平台 (easier to port) **← 最重要的優點

【題型 10】System Call 傳遞多參數的方法 (2013/2014 考)

題目：Describe an appropriate method used to pass "MANY" parameters to the OS during system calls.

標準答案：

使用 **memory block** (記憶體區塊) 或 **parameters marshalling** 方法：

將多個參數儲存在記憶體的一個區塊 (table) 中，然後把這個區塊的**記憶體位址**放入一個 register 傳遞給 OS。OS 再根據這個位址去讀取所有參數。這是 Linux 和 Solaris 採用的方法。這樣可以傳遞任意數量的參數，不受 register 數量限制。

【題型 11】OS 結構優缺點

Layered (分層架構) 優缺點 (2011/2016 考)

題目：Describe the pros and cons of layered operating system structure.

標準答案：

優點 (**Pros**)：

- 模組化 (**Modularity**)：各層只使用下一層的服務，隔離清楚。
- 易於除錯 (**Easy to debug / decouple**)：可以從最低層 (Layer 0) 開始逐層驗證；如果某層有問題，可以確定問題在該層。

缺點 (**Cons**)：

- 難以定義各層 (**Hard to define layers**)：很難決定各功能應放在哪一層，層次界限模糊。
- 效能差 (**Less efficient**)：每次服務請求都需穿越多層，每層都需 overhead；不如 monolithic 直接呼叫有效率。

Microkernel (微核心) 優缺點 (2012 考)

題目：Describe the pros and cons of the microkernel system structure.

標準答案：

優點 (**Pros**)：

- **可靠 (Reliable)** : 非必要服務以 user-level server 方式運行，若一個 server 崩潰不影響 kernel。
- **可移植 (Portable)** : 核心精簡，移植到新平台只需修改少量核心程式碼。
- **安全 (Secure)** : 可以更嚴格地控制 kernel 中的功能。

缺點 (Cons) :

- **效能開銷 (Performance overhead)** : 服務呼叫需在 user space 與 kernel space 間透過 message passing 溝通，頻繁切換造成效能大幅下降。

Modular kernel 優於 Layered 和 Microkernel (2013 考)

題目 : What are the advantages of the modular kernel approach over the layered and microkernel design techniques?

標準答案 :

比 **Layered** (分層) 好 : The modular kernel approach has higher efficiency since it doesn't have to go through so many layers. Any module can directly communicate with any other module without going through intermediate layers.

比 **Microkernel** (微核心) 好 : 不需要在 user space 和 kernel space 之間傳遞 message，模組在 kernel space 執行，避免了 message passing 的效能開銷，效率更高，同時仍能保持 OS 的大部分正確執行。

【題型 12】雙模式操作 (2012/2013 考)

題目 : What is the dual-mode operation and how does it ensure the proper execution of the operating system?

標準答案 :

雙模式定義 : 作業系統支援兩種執行模式 : **user mode (mode bit = 1)** 和 **kernel mode (mode bit = 0)**。某些對系統有害的操作只能在 kernel mode 執行。當發生 interrupt 或 trap 時，模式從 user 切換到 kernel；OS 完成處理後切回 user mode。

如何保護 OS : User mode 中的程式嘗試執行 **privileged instructions (特權指令)** 時，硬體會將其視為非法操作，觸發 trap 轉交 OS 處理，確保使用者程式不能直接存取 kernel 或重要的系統資源，防止 errant (錯誤) 或惡意程式破壞系統。

參、行程 (Processes)

【題型 13】★★ 行程狀態 (每年必考)

題目 (2011/2012/2013/2015/2016) : Show the state of a process in the following situations:

情境	答案
(a) after it is selected by a long-term scheduler	ready

情境	答案
(b) before it is selected by a short-term scheduler	ready
(c) after it is selected by a short-term scheduler	running
(d) before it invokes a blocking system call	running
(e) after it invokes a blocking system call	waiting
(f) after the blocking system call is complete	ready
(g) after it is interrupted by a hardware interrupt	ready
after its time slice expired (2015 Q5(g))	ready
after it is created (forked) by a parent (2015 Q5(a))	ready

⚠ 易錯重點：

- hardware interrupt 後 → **ready** (不是 waiting ! 行程沒在等任何 I/O 事件)
- time slice 到期後 → **ready** (不是 waiting ! 只是被搶占)
- blocking system call 之前 → **running** (行程還在跑，還沒呼叫)

【題型 14】Multiprogramming vs Time-sharing (2011/2012 考)

題目：Explain the difference between multiprogramming and timesharing.

標準答案：

- **Multiprogramming** (多程式處理)：同時在記憶體中保持多個 jobs，當一個 job 等待 I/O 時，CPU 切換到另一個 job，目的是最大化 **CPU 使用率**。
- **Time-sharing** (分時系統)：是 multiprogramming 的延伸 (**extension**)。CPU 以 time quantum 為單位快速切換，讓每個使用者都感覺有專屬的電腦，目的是提供 **互動式 (interactive)** 操作和短的 response time (通常 < 1 秒)。

差異：Multiprogramming 重視 CPU 利用率；Time-sharing 重視使用者互動性和 response time。Time-sharing is an extension of multiprogramming.

【題型 15】★★ fork() 輸出分析 (2012/2016 考)

2016 考題

```
int value = 5; // value is a global variable

int main(int argc, char *argv[]) {
    pid_t pid;
    pid = fork();
    if (pid > 0) {
        value += 15;
    }
}
```

```
    wait(NULL);
    printf("value = %d\n", value);
} else if (pid == 0) {
    value += 5;
    printf("value = %d\n", value);
}
}
```

(1) 輸出結果：

```
value = 10    (子行程先印, 5 + 5 = 10)
value = 20    (父行程後印, 5 + 15 = 20)
```

說明：fork() 後父子行程各有獨立的 value 副本（初始值均為 5）。子行程 value += 5 得 10 並印出；父行程等 wait(NULL) 完成後才印，value += 15 得 20。

(2) 移除 wait(NULL) 的影響：

- 父行程不等待子行程，父子**並行執行**，輸出順序**不確定**（**non-deterministic**）。
- 若父行程先於子行程結束，子行程變成**孤兒行程**（**orphan process**），由 init 接管。
- 子行程結束後，若父行程沒有呼叫 wait()，子行程的 PCB 殘留，成為**殭屍行程**（**zombie process**）。

2012 考題

```
int main(int argc, char *argv[]) {
    pid_t pid;
    int i = 0;

    pid = fork();
    i = i + 5;
    if (pid == 0) {
        i = i + 5;
        printf("i = %d\n", i);
    } else {
        wait(NULL);
        printf("i = %d\n", i);
    }

    pid = fork();
    i = i + 5;
    if (pid == 0) {
        i = i + 5;
        printf("i = %d\n", i);
    } else {
        wait(NULL);
        printf("i = %d\n", i);
    }
}
```

```
    return 0;
}
```

追蹤過程：

第一次 fork 後 ($i=0$)：

- 子行程： $i = 0+5 = 5$ (fork後共同執行)，再 $i = 5+5 = 10$ ，印出 $i = 10$
- 父行程： $i = 0+5 = 5$ ，wait 後印出 $i = 5$

第二次 fork (原父行程， $i=5$)：

- 新子行程： $i = 5+5 = 10$ ，再 $i = 10+5 = 15$ ，印出 $i = 15$
- 原父行程： $i = 5+5 = 10$ ，wait 後印出 $i = 10$

最終輸出 (確定順序)：

```
i = 10    ( 第一次 fork 的子行程 )
i = 5     ( 第一次 fork 的父行程，wait 後 )
i = 15    ( 第二次 fork 的子行程 )
i = 10    ( 第二次 fork 的父行程，wait 後 )
```

【題型 16】★★ Shared Memory 程式填寫 (2011/2012/2015 考)

計算 $\Sigma n!$ 版本 (2011 Q6，20分)

題目：Complete the following program which calculates $\Sigma(n=1 \text{ to } N) n!$. N is provided via command line.

```
int main(int argc, char *argv[]) {
    int N = atoi(argv[1]);
    pid_t pid;
    int segment_id;
    int *shared_memory;

    // 建立並附加共享記憶體
    segment_id = shmget(IPC_PRIVATE, N * sizeof(int), S_IRUSR | S_IWUSR);
    shared_memory = (int *) shmat(segment_id, NULL, 0);
    *shared_memory = 0; // 初始化為 0

    for (int i = N; i > 0; i--) { // 建立 N 個子行程
        pid = fork();
        if (pid < 0) {
            fprintf(stderr, "Fork Failed");
            exit(-1);
        } else if (pid == 0) { // 子行程
            // 計算 i! 並加到 shared memory
            int fact = 1;
            for (int j = i; j > 0; j--) fact *= j;
```



```
        *shared_memory += fact;
        shmdt(shared_memory);
        exit(0);
    } else {                                // 父行程
        wait(NULL);                        // 等子行程避免 race condition
    }
}

printf("the result is %d\n", *shared_memory);
shmdt(shared_memory);
shmctl(segment_id, IPC_RMID, NULL);
return 0;
}
```

計算 $2x+3y-5$ 版本 (2012 Q6 / 2015 Q9 · 15分)

題目：Complete the following program that calculates equation $2x+3y-5$.

```
int main(int argc, char *argv[]) {
    int x = atoi(argv[1]);
    int y = atoi(argv[2]);
    int i;
    pid_t pid;
    const int sh_size = 4;    // sizeof(int) = 4 bytes

    // 建立共享記憶體
    int segment_id = shmget(IPC_PRIVATE, sh_size, S_IRUSR | S_IWUSR);
    // 附加共享記憶體
    int *shared_memory = (int *) shmat(segment_id, NULL, 0);
    *shared_memory = 0;      // 初始化

    for (i = 2; i > 0; i--) { // 建立兩個子行程
        pid = fork();
        if (pid < 0) {
            fprintf(stderr, "Fork Failed");
            exit(-1);
        } else if (pid == 0) { // 子行程
            if (i == 2) *shared_memory += 2 * x; // 計算 2x
            if (i == 1) *shared_memory += 3 * y; // 計算 3y
            shmdt(shared_memory); // 分離
            exit(0);
        } else { // 父行程
            wait(NULL); // 等子行程
        }
    }

    *shared_memory -= 5; // 計算 -5
    printf("the result is %d\n", *shared_memory);
    shmdt(shared_memory);
    shmctl(segment_id, IPC_RMID, NULL);
}
```

```
    return 0;
}
```

【題型 17】★★ Shell 程式填寫 (2013 Q8 / 2016 Q5)

題目：Complete the following C++ program which serves as a simple shell interface. The shell accepts user commands and then executes each command in a separate process. The shell allows the child process to run in the background if the command ends with `&`.

```
int main(void) {
    char inputBuffer[80];    // buffer to hold command entered
    bool background;         // true if a command is followed by '&'

    while (cin >> inputBuffer) {        // 持續接受命令 ( 或用 !feof(stdin) )
        cout << endl << " COMMAND->";
        pid_t pid;

        // Step 1: 讀取命令並檢查是否以 '&' 結尾
        // 若 inputBuffer 最後一個字元是 '&'，設 background = true
        if (inputBuffer[strlen(inputBuffer)-1] == '&') {
            background = true;
            inputBuffer[strlen(inputBuffer)-1] = '\\0'; // 移除 '&'
        } else {
            background = false;
        }

        // Step 2: fork 子行程，根據 background 決定父行程是否等待
        pid = fork();
        if (pid == 0) {                // 子行程
            exec(inputBuffer);         // 執行使用者命令
        } else if (pid > 0) {          // 父行程
            if (background == false)
                wait(NULL);            // 前景執行：等子行程結束
            // 背景執行 ( background == true )：不等待，直接繼續迴圈
        }
        exit(0);                      // 子行程執行完後結束
    } // end of while
} // end of main
```

重點說明：

- `background = true`：子行程在背景執行，父行程不呼叫 `wait()`，繼續接受下一個指令
- `background = false`：父行程呼叫 `wait(NULL)` 等待子行程結束 (前景執行)

肆、執行緒 (Multithreaded Programming)

【題型 18】Thread 為何比 Process 便宜 (2015/2016 考)

題目：Explain why allocating memory for process creation is costly as compared with thread creation.

標準答案：

Every process contains program code, stack, heap, and more in memory. If two or more processes need to use the same data, the data will be copied to the other process's stack or heap, **costing more memory space**.

Different threads can share memory with each other without more memory spaces. Thread 只需建立自己的 Thread ID、Program Counter、registers 和 stack，共享同一行程的 code/data/heap/open files，不需複製整個位址空間。在 Solaris 中，Thread 建立比 Process 快約 **30 倍**。

【題型 19】★ Many-to-One vs One-to-One 執行緒模型 (每年必考)

題目 (**2013/2015/2016**)：Compare and contrast the many-to-one and one-to-one thread models. (或：Describe the pros and cons of the one-to-one thread model.)

標準答案：

Many-to-One (多對一)：

The many-to-one model has a kernel thread that allocates many user threads. It can be quick in executing because it will not invoke system calls.

- ☒ 優點：Thread 管理在 user space，效率高
- ☒ 缺點：若一個 user thread 做 blocking system call，所有 **threads** 都會被阻塞；無法在多處理器平行執行

One-to-One (一對一)：

The one-to-one model can fix that [blocking] problem but is limited in number.

- ☒ 優點：一個 thread 阻塞不影響其他；可在多處理器真正平行執行
- ☒ 缺點：每建立一個 user thread 就要建立一個 kernel thread，**overhead** 大；系統中 kernel thread 數量可能受到限制

【題型 20】★ Scheduler Activations (2011/2012/2014 考)

2011 Q7(a)：Scheduler Activation 要解決什麼問題？

題目：What is the problem of many-to-many thread model that scheduler activation tries to resolve?

標準答案：

In the many-to-many model, when a user thread performs a blocking system call (e.g., I/O), the kernel will block the entire process including all other user threads mapped to the same kernel thread. The scheduler activation mechanism addresses this by allowing the kernel to notify the thread library about blocking events, so the library can schedule other user threads.

(問題：M:M 模型中，一個 user thread 做 blocking I/O，kernel 阻塞整個行程，其他 user thread 無法繼續執行)

2011/2012 Q7(b)：Scheduler Activation 的運作

題目：Describe the operation of scheduler activation.

標準答案：

Scheduler activation 使用 **LWP (Lightweight Process，輕量級行程)** 作為 kernel 和 thread library 之間的虛擬處理器。運作流程如下：

1. User thread 準備執行 **blocking system call**
2. Kernel 偵測到此 thread 即將阻塞，透過 **upcall (向上呼叫)** 通知 thread library
3. Thread library 的 **upcall handler** 被呼叫，它：
 - 儲存被阻塞 thread 的狀態
 - 在可用的 LWP 上排程另一個可執行的 **user thread**
4. 當 blocking call 完成，kernel 再次發出 **upcall**，通知 thread library
5. Thread library 將原來被阻塞的 thread 標記為 **ready**，擇機重新排程執行

伍、CPU 排程 (CPU Scheduling)

【題型 21】可搶占排程的 4 種情況 (2014 考)

題目：描述 preemptive 和 nonpreemptive scheduling 的差異，並列出 CPU scheduling decisions 發生的 4 種情況，哪些屬於 preemptive？

標準答案：

情況	描述	類型
①	行程從 running 進入 waiting (如 I/O request)	非搶占 (必須選新的)
②	行程從 running 進入 ready (如 interrupt 發生)	可搶占 ★
③	行程從 waiting 進入 ready (如 I/O 完成)	可搶占 ★
④	行程 terminated	非搶占 (必須選新的)

情況 1 和 4 是非搶占排程 (行程自願放棄 CPU)。情況 2 和 3 是**可搶占排程**的關鍵：OS 可強制取走 CPU 使用權。

【題型 22】Priority Scheduling 的飢餓問題 (2015 Q7 考)

題目：Design a solution to avoid the starvation problem of priority scheduling algorithms.

標準答案：

When there are infinite processes with high priority, it is possible that processes with lower priority will never be executed. The solution is called **aging** (老化) .

Aging 的做法：系統維護一個隨時間不斷增加的數值（稱為 "time"）。若一個行程的等待時間（age）足夠長（old enough），它的優先權會持續上升，直到高到足以被選中執行為止。這樣確保了即使系統中持續有高優先權的行程，低優先權的行程最終也能被執行。

【題型 23】★★ 多層反饋佇列甘特圖（2014/2015 考）

題目（2015 Q8）：Consider processes P1 and P2:

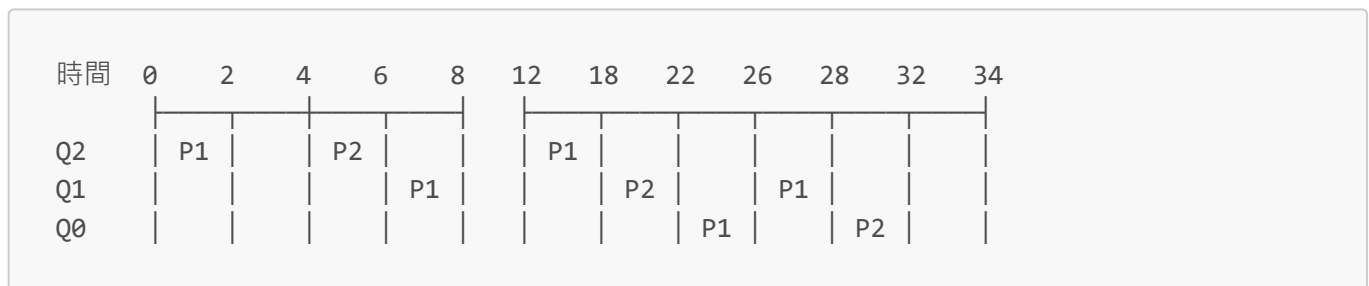
- P1：到達 $t=0$ ，CPU burst=8ms，I/O=4ms，CPU burst=8ms
- P2：到達 $t=2$ ，CPU burst=2ms，I/O=2ms，CPU burst=16ms

使用以下 **preemptive multilevel feedback-queue** 排程（高層搶占低層，被搶占降層）：

- Q2：Round-Robin，time quantum = 4ms
- Q1：Round-Robin，time quantum = 6ms
- Q0：FCFS

(a) 畫出 Gantt chart (b) 計算平均等待時間

標準答案（Gantt Chart）：



詳細時間軸：

時間	執行	隊列	說明
0-4	P1	Q2	P1 執行 4ms，quantum 到，降至 Q1
4-6	P2	Q2	P2 在 $t=2$ 到達 Q2，執行 2ms 完成，去 I/O（ $t=6$ 完成）
6-12	P1	Q1	P1 在 Q1 執行 6ms quantum，降至 Q0（剩 2ms burst）
8	P2 I/O 完成	Q2	P2 回到 Q2，但 P1 在 Q1 中， $Q2 > Q1$ ，P2 搶占 P1
8-12	P2	Q2	P2 在 Q2 執行 4ms quantum，降至 Q1

（根據答案卷）最終：

- P1 等待時間 = $(2+10) = 12\text{ms}$
- P2 等待時間 = $(2+2+8) = 12\text{ms}$
- 平均等待時間 = $(12+12) \div 2 = 12\text{ms}$

【題型 24】排程等待時間計算

公式：

等待時間 (Waiting Time) = 在 ready queue 等待的總時間
 = 完成時間 - 到達時間 - CPU burst 總長度
 (注意：I/O 等待時間不算在內)

平均等待時間：所有行程等待時間的平均值。

附錄：常見錯誤整理

容易答錯的題目	錯誤答案	正確答案	原因
hardware interrupt 中斷後的狀態	waiting	ready	中斷只是搶占 CPU，行程沒在等任何 I/O 事件
time slice 到期後的狀態	waiting	ready	只是被搶占，行程沒在等任何事件
blocking system call 呼叫之前的狀態	waiting	running	還沒呼叫，行程還在 CPU 上執行
fork() 在子行程的回傳值	1	0	父行程才回傳子 PID，子行程回傳 0
分離共享記憶體函數	shmctl	shmdt	shmdt 是 detach，shmctl(IPC_RMID) 是移除
Linux 使用的執行緒模型	Many-to-Many	One-to-One	Linux 每個 user thread 對應一個 kernel thread
mode bit = 0 代表	user mode	kernel mode	0 = kernel，1 = user
Time-sharing 和 Multiprogramming 的關係	相同	Time-sharing 是 Multiprogramming 的延伸	

附錄：考試年份題目對照索引

題型	2011	2012	2013	2014	2015	2016
Interrupt-based vs Busy-waiting	Q1(a)	Q1(a)意	Q1(a)	Q1(a)	Q1(b)相關	Q1(a)
中斷七步驟流程圖	Q1(b)	Q1(b)	Q1(b)	Q1(b)	Q1(b)	Q1(b)
Interrupt vector 好處	Q1(c)	—	—	Q1(c)相關	—	—

題型	2011	2012	2013	2014	2015	2016
Device Controller vs Driver	—	—	Q2	Q3	Q1(a)	Q1(d)相關
API 呼叫 System Call	Q3	Q2(a)	Q4(a)	Q1(a)	Q4(a)	Q2(a)
為何偏好 API	Q3	Q2(b)	Q4(b)	—	Q4(b)	Q2(b)
傳遞多參數方法	—	—	Q4(c)	—	—	—
Layered 架構優缺點	Q4	—	—	—	—	Q2(c)
Microkernel 優缺點	—	Q3	—	—	—	—
Modular 優於其他	—	—	Q5	—	—	—
CLI 兩種方式	—	—	—	—	Q3	—
高階語言優點	—	—	—	—	Q2	—
Dual-mode	—	Q1(d)	Q3	—	—	—
Multiprogramming vs Time-sharing	Q2	Q1(c)	—	—	—	Q1(c)
行程狀態 (7題)	Q5	Q4	Q6	—	Q5	Q3(a)
fork() 輸出分析	—	Q5	—	—	—	Q3(b)
Shared memory 程式	Q6	Q6	—	相關	Q9	—
Shell 程式填寫	—	—	Q8	—	—	Q5
Thread vs Process 便宜	—	—	—	—	Q6(a)	Q4(a)
Many-to-One vs One-to-One	—	Q7(a)	Q7	—	Q6(b)	Q4(b)
Scheduler Activation	Q7(b)	Q7(b)	—	Q(?)	—	—
排程甘特圖	—	—	—	Q8	Q8	—
Priority Starvation/Aging	—	—	—	—	Q7	—